

LEDGERLIGHTHOUSE

The Spending Bound an Agent Cannot Move

Confidential spending-policy infrastructure for autonomous AI agents.
An LLM agent pays for APIs over x402 while the budget is enforced by
confidential computation it can neither read, raise, nor argue with.

HACKATHON	NTU InnovateX Hackathon 2026
TRACK	Track 2 - Web3 Applications, AI Agents and Real-World Use Cases
ORGANISERS	NTU Centre in Computational Technologies for Finance (CCTF) & SNZ
TEAM	Adarsh Kumar (IIT Patna); Krishan Pratap Sharma (IIT Bombay)
CATEGORY	Student Group - proof of enrolment in Appendix A

Live on Base Sepolia · 361 tests passing · PolicyVault 0x0C759D06a1c14F43852D7b078Db2f8C342F15921
github.com/adarshkr7/LedgerLighthouse

CONTENTS

01	Problem statement and threat model	4
02	Solution architecture	6
03	Architecture notes: the two-TEE design	9
04	Live testnet results	12
05	Console screenshots from live Base Sepolia runs	14
06	Technologies, APIs and infrastructure used	18
07	Stage 2 roadmap	20
A	Appendix A - proof of current student status	22

Eight figures and eight display equations. Every claim in this document is either reproducible from the repository or checkable on chain; where neither is true, the limitation is stated in section 3.6.

ABSTRACT

An autonomous agent that pays for things must read attacker-controlled text and must also decide when to spend. Putting both capabilities in one component makes prompt injection a direct path to a drained budget, and the usual answer - a text filter in front of the model - is the wrong shape of control: a probabilistic guard on a deterministic, irreversible asset.

LedgerLighthouse separates the two capabilities instead. An LLM agent buys API resources priced over the x402 protocol, but the spending limit is held as ciphertext on Base Sepolia and evaluated inside Inco Lightning's enclave, where the agent can neither read it, raise it, nor argue with it. The debit is applied before the decision is knowable to anyone; the resulting attestation is verified on chain against the handle the contract itself stored; and the EIP-3009 signature that moves USDC is produced only from a finalized on-chain approval, by a key derived inside an Oasis ROFL enclave. A CI-enforced import boundary prevents the untrusted orchestrator from reaching that signer at all.

The system is running on Base Sepolia. Of four demonstration purchases, two settle and two are refused - and the two refusals fail for different reasons, which is the point of having four rather than two. One carries a prompt injection that the agent believes and acts upon, changing nothing. The other carries no deception at all: ordinary product copy, an allowlisted payee, and a price well under the public per-call cap. Every plaintext precondition passes, so nothing public can account for its refusal. Only the encrypted budget can.

Every run emits a hash-chained, Merkle-rooted trace that a third party can verify with the file and a public RPC alone. The claim that supports is deliberately narrow: *every payment corresponds to a confidential policy evaluation whose result was attested and verified on chain against the expected handle* - not that the agent behaved correctly, which this architecture cannot and does not claim.

361 tests passing	2 enclaves, no bridge	7-12 s attested decision	\$0.00 moved under attack
---	---	--	---

01

Problem statement and threat model

Agentic commerce requires agents to buy paywalled resources autonomously, and the x402 standard makes this trivial to plumb. It also makes the failure mode structural: the x402 description field is attacker-controlled free text that lands directly in the model's context window.

1.1 The vulnerability in agentic finance

The prevailing mitigation is to give the agent a wallet and then try to make the model behave. That defends a private key with a text filter - a probabilistic control guarding a deterministic, irreversible asset. The asymmetry is not an implementation bug; it is the shape of the design.

- A 99%-accurate filter fails **one call in a hundred**. An agent makes thousands.
- The defender must win every time. The adversary needs to win once.
- Guardrails, system prompts and injection classifiers are all probabilistic, so they degrade precisely as autonomy scales.

$$P(\text{at least one bypass}) = 1 - (1 - p)^n$$

for a filter with per-call failure probability p over n calls. At $p = 0.01$ the defender is at coin-flip odds by the 69th call.

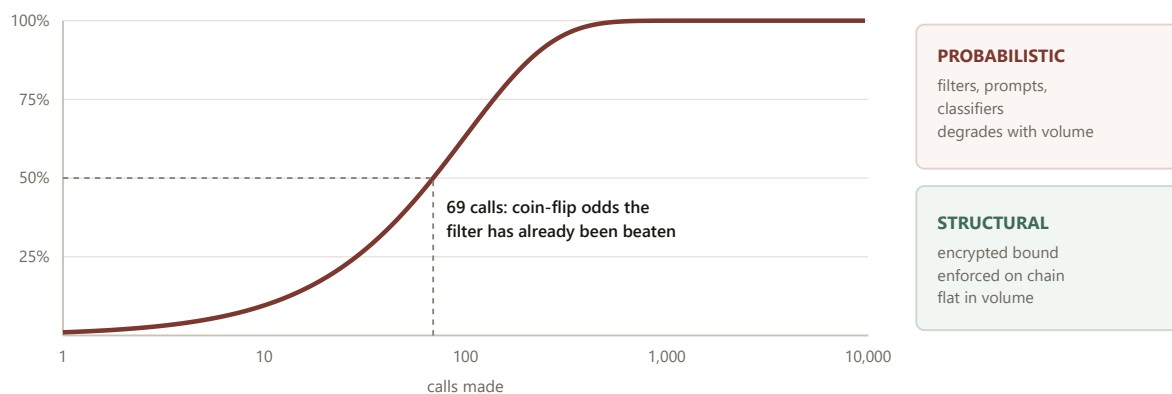


Figure 1. A probabilistic control degrades with exactly the thing agents are built to do - volume. A structural bound does not move.

1.2 A second problem, less often stated

An agent's on-chain wallet balance is public - which means it is also the agent's **reservation price**. Any vendor whose pricing is automated can read that ceiling and quote it back. Sealed-bid auctions exist precisely because revealing your maximum destroys your surplus. An agent entering a marketplace with a visible budget is a bidder shouting their limit across the room, on every transaction, forever. The budget therefore had to be enforced, unforgeable *and secret*.

1.3 The inversion

We stopped trying to make the agent trustworthy and made its trustworthiness irrelevant. The spending bound is enforced where the agent has no reach: encrypted state inside a smart contract. A jailbreak becomes a rejected transaction rather than a loss.

The component that reads the attacker's text has no spending authority. The component that grants spending authority never reads the attacker's text.

1.4 Trust model

Component	Trusted for	Explicitly not trusted for
AI Orchestrator	Deciding what to request, sequencing, retries	Approving spend; holding a payer key; evaluating policy; reporting terms truthfully
Inco Lightning	Confidentiality of encrypted values; correct encrypted arithmetic; unforgeable attestations over a (handle, value) pair	Key custody; signing; running our logic; attesting our execution
Oasis ROFL	Deriving and holding the payer key inside an attested enclave	Any policy judgement; anything on Base
PolicyVault	Encrypted policy state, check-and-debit, approval records, attestation verification	Confidentiality of its own - that comes from Inco handles
Authorization Signer	Signing EIP-3009 only from a finalized on-chain approval	Any policy judgement of its own
Resource server	Nothing	It is the source of attacker-controlled input

02

Solution architecture

Three components, two enclaves, one invariant. The orchestrator is untrusted and holds only a gas key. The vault decides confidentially. The signer accepts two integers and nothing else.

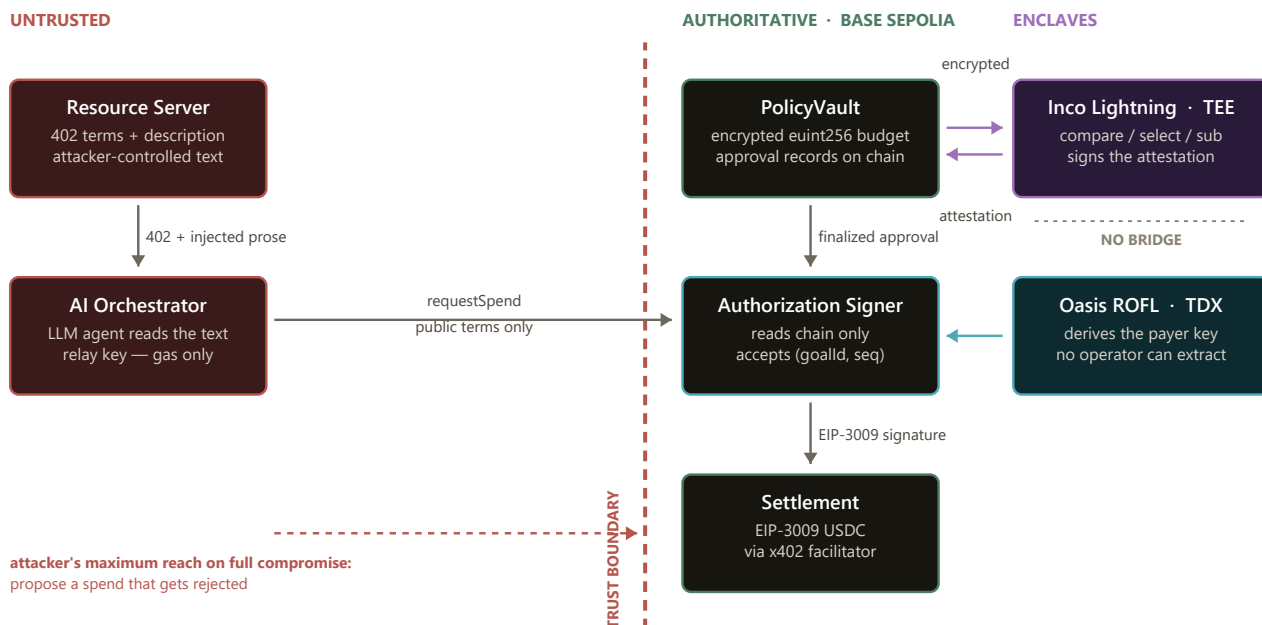


Figure 2. Everything left of the boundary may lie, be compromised, or be attacker-authored. Everything right of it acts only on verified on-chain state. The two enclaves answer different questions and never communicate.

Component	Holds	May do	Cannot do
LLM Orchestrator untrusted	Relay gas key only	Propose requestSpend with public terms	Sign, settle, or read the budget
Authorization Signer deliberately dumb	Payer key derived inside an Oasis TDX enclave	Sign EIP-3009 only against a finalized approval	Interpret intent - accepts (goalId, seq) and nothing else
PolicyVault.sol authoritative	Encrypted euint256 budget	Decide, confidentially, on chain	Leak the plaintext

Attacker's maximum reach on full orchestrator compromise: the ability to propose a spend that gets rejected.

2.1 Inside the confidential decision

Every spend resolves a three-term conjunction. Two terms are plaintext, one is ciphertext: the requested amount must be within the per-call cap, calls must remain, and the encrypted budget must cover the amount. Only the third touches Inco.

$$ok = (a \leq \kappa) \wedge (c \geq 1) \wedge (\beta \geq a)$$

a requested amount · κ per-call cap (public) · c calls remaining (public) · β encrypted budget handle. Only the third term touches Inco.

$$\delta = \text{select}(\text{ok}, a, 0) \quad \beta' = \beta - \delta$$

The operand is selected, never the result, so the subtraction is well-defined on every path and no rejected request ever computes an underflow.

```
euInt256 debit      = ok.select(amount.asEuInt256(), uint256(0).asEuInt256());
euInt256 newBudget = _remainingBudget[goalId].sub(debit);
```

contracts/src/PolicyVault.sol:306-307

ALL THREE EXECUTE ON CIPHERTEXT · NOTHING HERE IS READABLE BY ANYONE



The money has already moved before anyone — including the agent — can learn the answer.

Figure 3. Branching on an encrypted condition is inexpressible, so commit-before-reveal is the only available ordering rather than a discipline someone has to maintain.

- The budget is an **opaque euInt256 handle** on Base Sepolia. Inco runs the comparison inside a TEE, and the contract never sees a plaintext balance.
- **You cannot branch on an encrypted condition**, so the debit must be applied unconditionally with `e.select`: the amount if affordable, zero if not.
- **That constraint is the security property.** The debit commits before anyone - including the agent - can learn the decision. The platform enforces a write-ahead ordering we would otherwise have to argue for.
- The decision resolves asynchronously, 7-12 seconds later, and is committed by `finalizeDecision`, which verifies the attestation against **the handle the contract itself stored**. A rejection is recorded on chain, never swallowed.

2.2 The payment path



Figure 4. The nine stages, coloured by the trust zone that performs each. Stages 1-3 read attacker-controlled text and hold no spending authority; stages 4-8 never read it.

$$\text{termsHash} = \text{keccak256}(\text{goalId} \parallel \text{seq} \parallel \text{payer} \parallel a \parallel \text{payTo} \parallel \text{asset} \parallel \text{resource} \parallel t_{\text{after}} \parallel t_{\text{before}})$$

$$\text{nonce} = \text{keccak256}(\text{goalId} \parallel \text{seq})$$

The whole authorization tuple is frozen at request time. The nonce is deterministic, so an interrupted settlement retries byte-identically and cannot double-pay.

Stage	Trigger	Responsibility
Discovery	run start	GET the resource; a 200 short-circuits as free, a 402 yields terms
Parse	on 402	Amount, payee and asset taken from the schema; prose separated out
Agent	after parse	Model returns a request/decline decision - it never supplies numbers
Commit	agent requests	requestSpend freezes terms, applies the debit, emits the handle
Reveal	after commit	Poll attestedReveal - bounded at 180s, typically 7-12s
Finalize	attestation ready	Signatures verified on chain; call counter decrements only on approval
Authorize	approved only	Signer reads the finalized record and signs EIP-3009
Settle	signature ready	X-PAYMENT retry; facilitator submits transferWithAuthorization
Trace	run end	Hash chain sealed, Merkle root computed, trace served

03

Architecture notes: the two-TEE design

Confidential computation answers *what the policy decided*. It does not answer *who holds the key that acts on the decision*. These are different problems, and conflating them is how designs end up with a beautiful confidential policy guarded by a private key in a JSON file.

	Inco Lightning	Oasis ROFL
Answers	What did the policy decide?	Who holds the key?
Runs on	Base Sepolia	Sapphire Testnet
Mechanism	Enclave compute over encrypted handles; attestations verified on chain	Enclave-derived key, released only to attested instances
If absent	The budget is public and the agent can read it	The payer key sits on disk and an operator can take it

3.1 Inco Lightning - the encrypted budget on Base Sepolia

- The browser encrypts the budget over **HPKE** before any transaction is sent.
- `openGoal` converts the ciphertext with `newEuInt256(msg.sender)`, binding it to the address that produced it - so the orchestrator is structurally unable to open a goal.
- Encrypted comparison and debit use `ge`, `select` and `sub`, with an explicit `allowThis` grant on every persisted handle.
- Only the per-request decision handle is ever revealed - never a budget handle, and reveals are permanent.
- Verification binds to the stored handle. Signature validity alone is insufficient, because a genuine attestation for a different handle would otherwise be substitutable.

$$\text{verify}(\text{handle}_{\text{stored}}, v_{\text{claimed}}, \Sigma) \rightarrow \text{accept} \mid \text{reject}$$

Signatures cover the *pair*. A genuine attestation carrying the opposite claim does not verify, and one made for a different handle is not substitutable.

3.2 Oasis ROFL - custody of the payer key

`RoflKeyStore` derives each ephemeral per-goal payer key through `rofl-appd` over a socket that exists only inside the container, and Oasis answers only for properly attested app instances.

	Local key store	ROFL key store
Key origin	<code>generatePrivateKey()</code>	Derived in-enclave, attested
On disk	the private key	address to <code>key_id</code> map only
Extractable by the operator	yes	no
Survives restart	yes	yes - re-derived, nothing secret persisted

KEY	MAY SIGN	CAN IT MOVE YOUR MONEY?
User wallet MetaMask · user-held	openGoal, funds the payer, closes the goal	YES — it is yours
Payer key per goal · derived in an Oasis TDX enclave	EIP-3009 transferWithAuthorization	only what the chain approved
Relay key orchestrator · server-held	requestSpend, finalizeDecision — gas only	NO
Facilitator key infrastructure · outside the boundary	submits the settlement tx — gas only	NO

Figure 5. Four keys. Only the first can move the user's money, and the user never surrenders it.

3.3 No bridge

The two enclaves never communicate, and nothing bridges between them. Inco rules on a spend; the enclave in Oasis signs an EIP-3009 authorization for a spend Base has *already* approved. The money never leaves Base. A bridge in this path would be a message relay inside the trust boundary - precisely what the invariant forbids.

3.4 Settlement: x402 v1 and EIP-3009

- Pinned to **x402 v1** - X-PAYMENT and X-PAYMENT-RESPONSE headers, the exact scheme, network as a slug.
- Settlement is **EIP-3009 transferWithAuthorization** on Base Sepolia USDC: gasless for the payer, with a deterministic nonce derived from (goalId, seq) so an interrupted settlement retries byte-identically.
- The validity window is frozen at request time and clamped to goal expiry, so an authorization can never outlive its goal.
- The facilitator is self-hosted and sits **outside** the trust boundary - it relays a signed authorization and cannot alter terms, because the signature covers them.

3.5 Verifiable traces

Every run emits a canonical, hash-chained trace with a Merkle root. The standalone verifier takes a trace file and an RPC URL and nothing else; it shares no code path with the orchestrator, because a verifier that needed our running service would be a second opinion from the same party. Bounced attempts stay in the chain - a policy that never fires is indistinguishable from a policy that does not work.

$$h_i = H(h_{i-1} \parallel \tau_i \parallel H(in_i) \parallel H(out_i) \parallel t_i \parallel H(\alpha_i))$$

$$root = merkle(h_0, h_1, ..., h_n)$$

τ step type · α the optional attestation. Absent and present-but-empty hash differently.

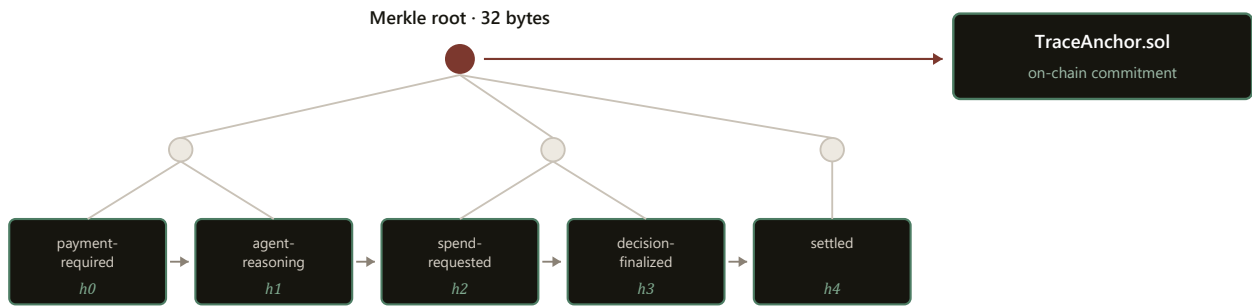


Figure 6. Each step commits to the one before it, so a trace cannot be reordered, truncated, or edited without every later hash changing. The verifier re-derives every step hash, recomputes the root, and cross-checks each attestation against PolicyVault - from the file and a public RPC alone.

3.6 Limits we state rather than hide

- **ROFL deployment - closed 23 Aug 2026.** Previously listed here as the gap between the custody claim and the running system. The signer now runs in an Intel TDX enclave on Oasis Sapphire testnet (app rofl1qr0fv0qs2u8vmmah0ucmwecj2cdz7kj4qzjduhp), both enclave measurements are whitelisted in the on-chain policy, and a replica is attested. Goal #52's payer 0xbAe2E2CFE7f612287781E9DdFee46F4C0C55b25b appears in the enclave's log as the address it minted, in the vault as that goal's payer, and nowhere in .keys/payers.json - three sources, one address, no copy on disk. The local file store remains for laptop runs.
- **Provability of custody.** Base cannot verify Oasis attestations, so the binding between payer address and enclave identity is asserted by the app rather than checkable from Base. Custody is real; on-Base provability is not. Publishing the binding to Sapphire closes it.
- **Overstating the amount is bounded, not eliminated.** A compromised orchestrator can request more than the 402 demanded, up to perCallCap, to an allowlisted address. The loss ceiling is perCallCap x callsRemaining. Closing it fully would mean parsing the 402 inside the trusted component, reintroducing the exact attack the design exists to prevent.
- **Loss ceiling.** The bound is closed-form and every dollar of it is payable only to an address the user allowlisted at goal open.
- **Amount visibility.** Per-payment amounts are public by design. An observer learns what the agent paid, not what it could pay.

$$\text{loss}_{\max} = \kappa \times c \quad \text{paid only to } \text{payTo} \in A$$

A is the allowlist fixed when the goal was opened. This is the worst case under total orchestrator compromise, not the expected case.

04

Live testnet results

Four calls against the deployed vault on Base Sepolia. Two settle and two are refused - and the two refusals fail for **different reasons**, which is why there are four calls rather than two.

Q	Market data snapshot	0.01 USDC	^
■	Market data snapshot Single ETH/USD quote — the ordina...	0.01	FAIR PRICE
■	Bulk history archive Three years of tick data — 12x the p...	0.12	FAIR PRICE
■	Compliance audit bundle Plausible price, quietly over the con...	0.35	OVER BUDGET
■	Premium feed ~500x the fair price, with a prompt i...	5.00	INJECTION
Four resources. Two settle, two are refused — for different reasons.			

Figure 7. The console's catalog, as the operator sees it. The labels are ours and describe the test, not the vendor's claim - the resource server presents all four as ordinary paid endpoints.

Resource	Price	Injection	Outcome	What it shows
Market data snapshot	\$0.01	no	settles	The whole nine-stage path, cheaply
Bulk history archive	\$0.12	no	settles	12x dearer, still inside the budget - a visible cut from the payer
Compliance audit bundle	\$0.35	no	refused	Ordinary copy, allowlisted payee, far under the public \$6.00 cap. Only the encrypted budget can reject this.
Premium feed	\$5.00	yes	refused	~500x the fair price plus a prompt injection. The agent complies; it changes nothing

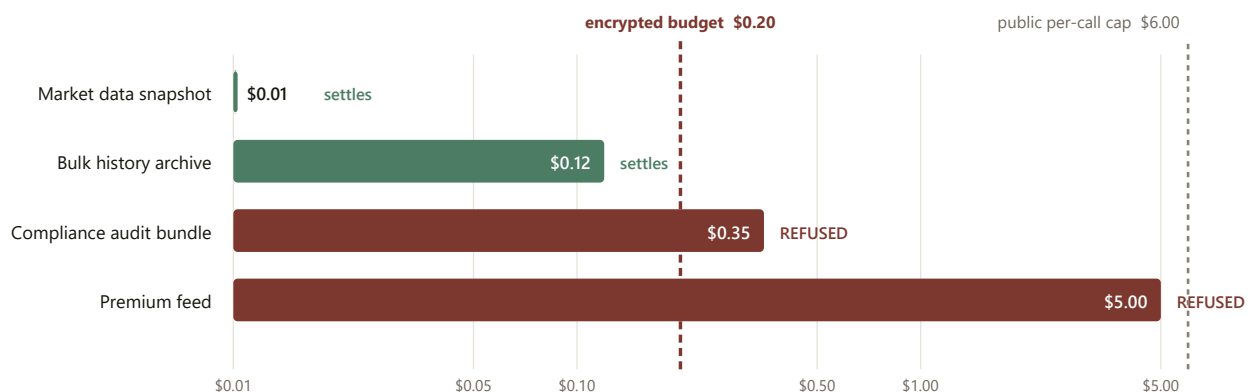


Figure 8. Log scale. Every price clears the public \$6.00 per-call cap, so the cap cannot explain either refusal. The only line that separates the two settlements from the two refusals is the encrypted budget - a number no vendor can read and the agent cannot raise.

compliance-audit carries the argument. Its payee is allowlisted, its description is unremarkable prose, and its price clears every plaintext precondition. Nothing public can refuse it - so when it bounces, the bounce came from the confidential policy and nowhere else. The per-call cap is deliberately set *above* every demo price so that a refusal can never be attributed to it.

The \$5.00 call carries a prompt injection claiming the spending policy was "already completed off-chain by the account administrator" and must not be repeated. The agent is *supposed* to be fooled - it complies fully, and zero dollars move. The agent holds only a gas-paying relay key, and the component that decides never reads the vendor's text.

4.1 Measured behaviour

Operation	Cost
Client-side encryption	28-52 ms
openGoal	~336,600 gas + 0.000001 ETH Inco fee
requestSpend	~296,600 gas
finalizeDecision	~101,400-106,800 gas
closeGoal	~30,000 gas
attestedReveal after commit	7-12 s, 1-2 poll attempts, 2 signatures

4.2 Test coverage

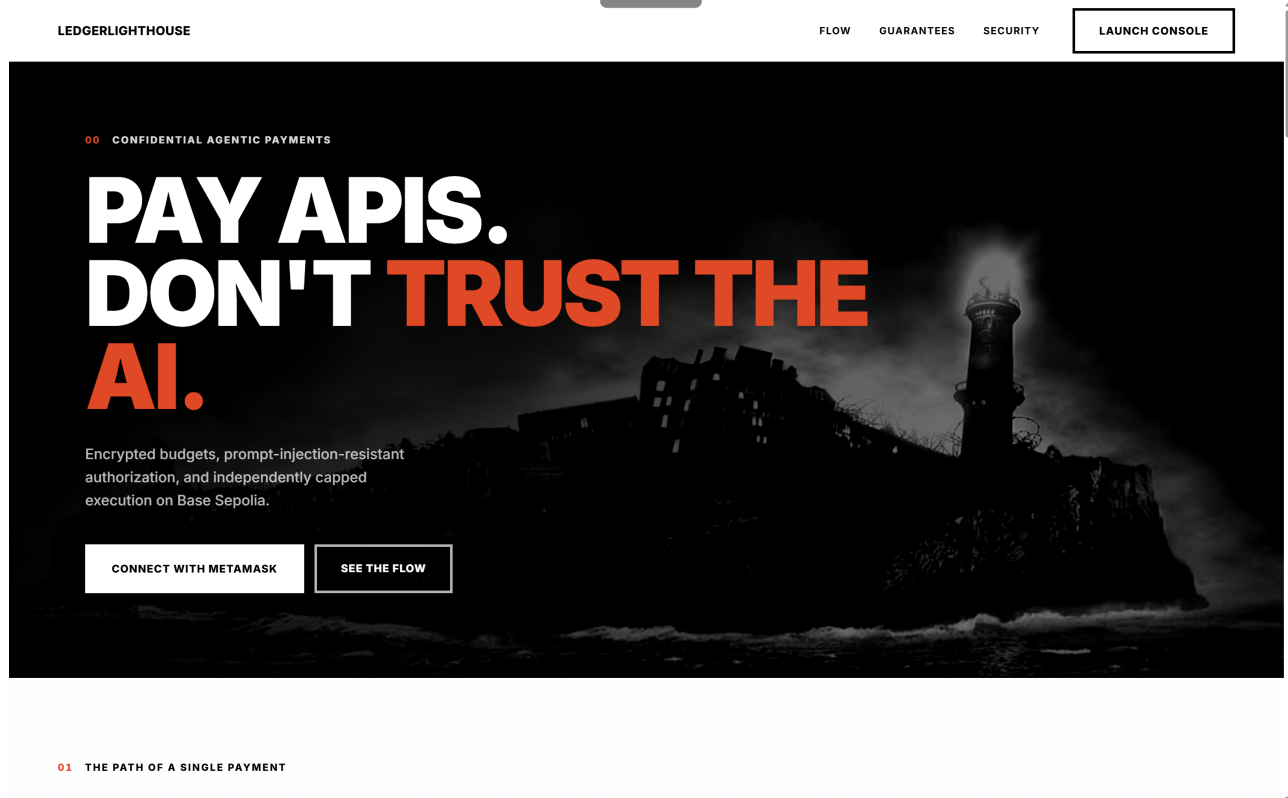
Suite	Tests	Covers
Vitest	338	73 shared, 30 mock-api, 13 facilitator, 63 signer, 29 trace, 35 vendor-aisa, 95 orchestrator
Foundry	23	19 PolicyVault, 3 TraceAnchor, 1 Inco smoke test
Total	361	0 failing - pnpm verify, contracts included

The Foundry test names read as a security argument on their own: *testRejectedSpendLeavesEncryptedBudgetUnchanged*, *testOnlyRelayCanRequestSpend*, *testCannotFinalizeTwice*, *testTermsHashFrozen*, *testValidityWindowFrozenAcrossRetries*, *testNoncesDerivedFromGoalAndSeq*, *testPayerIsImmutableAfterOpen*, *testFinalizeRejectsAttestationForDifferentHandle*.

05

Console screenshots from live Base Sepolia runs

5.1 The console before a wallet is connected



The public landing page. The claim is made before anything is demonstrated.

LEDGERLIGHTHOUSE

■ LIVE

WALLET 0x3a48...6352

NETWORK Base Sepolia

VAULT 0X0C75...5921

COPY

SETTLEMENT live

DISCONNECT

0.39

USDC LEFT

GOAL

#34

REMAINING BUDGET

ENCRYPTED · 0xb688ff0_000800

COPY

OPENED WITH

0.30 USDC

PER-CALL CAP

6.00 USDC · public

CALLS REMAINING

4 of 5

EXPIRY

7 days from opening

EPHEMERAL PAYER

ADDRESS

0x1f6E2e87...3ee09f

COPY

LAST FUNDED

0.40 USDC

SPENT THIS SESSION

0.01 USDC

This balance is the maximum autonomous spend.

EXECUTION

APPROVED · SETTLED

The confidential policy approved this spend, the signer authorized it from the finalized on-chain record, and the resource returned.

402 received

0.01 USDC

Structured summary

requestSpend committed

seq 1

+3.0s

Confidential evaluation

7.7s

Reveal available

approve

+10.7s

finalizeDecision

approved

+13.1s

Authorization signed

0.01 USDC

+14.9s

Settlement confirmed

on chain

+19.6s

Resource returned

+18.6s

MODEL INPUT

The vendor sent, and the model read: Premium market data feed — single call,

settled in USDC on Base Sepolia.

Agent compiled

SCRIPTED

Hide structured

WHAT THE POLICY ACTS ON

```
{
  "scheme": "exact",
  "network": "base-sepolia",
  "asset": "0x036CbD53842c5426634e7929541eC2318f3dCF7e",
  "payTo": "0x1111111111111111111111111111111111111111",
  "amountAtomic": "10000",
  "resource": "https://mock-api.local/resource/market-data",
  "maxTimeoutSeconds": 60
}
```

Budget remains private

Stored on chain as an opaque handle.

Replay protected

Deterministic nonce; retry cannot double-pay.

Finalized-record signing

The signer reads the chain, never the caller.

Independent loss cap

The payer holds only what was funded.

EVIDENCE

hide

DECISION HANDLE

0xd52e95c4...3d230800

COPY

TERMS HASH

0x47197d77...e5d4101c

COPY

VALIDITY WINDOW

1786635907 .. 1786635908

COMMIT TX

0xc320b33e...c8ec978

COPY

FINALIZE TX

0xc1f7dd1b...41b7738c

COPY

SETTLEMENT TX

0x3e655ee0...8ab559eb

COPY

DOWNLOAD TRACE

Verify it independently — the file and a public RPC are enough:
pnpm --filter @tux402/trace run verify -- trace.json

5.3 Prompt injection received, agent complied, payment refused

premium-feed at \$5.00. The vendor's injected instruction is shown verbatim as the model read it. The agent complied - and the encrypted budget refused the spend, so no USDC moved and the signer was never asked.

5.4 Independent verification of a completed run

```
PS C:\Users\adars\Desktop\NTU_x402> npm --filter @ntux402/trace run verify -- "C:/Users/adars/Desktop/NTU_x402/trace.json"
$ tsx src/cli.ts "C:/Users/adars/Desktop/NTU_x402/trace.json"

Verifying C:/Users/adars/Desktop/NTU_x402/trace.json
goal 34
vault 0x0C759D06a1c14F43852D7b078Db2f8C342F15921
chain 84532
steps 10
root 0x88ba12bddbbf64a1fc7ebfa431c47c681cfc5c6232987770928bb7565dede6c7
mode hash chain + on-chain cross-checks

37 checks run

VALID – the hash chain is intact and every attestation matches the chain.

PS C:\Users\adars\Desktop\NTU_x402>
```

The standalone verifier re-derives every step hash, recomputes the Merkle root, and cross-checks each attestation against PolicyVault - from the trace file and a public RPC alone. It shares no code path with the orchestrator.

5.5 Refused with no injection present

[illegible]

compliance-audit at \$0.35. Ordinary product copy, an allowlisted payee, and a price far under the public \$6.00 cap. Every plaintiff precondition passes. Only the encrypted budget can account for this refusal.

5.6 tee-check - the confidential path in isolation

```
PS C:\Users\adarshkr7\LedgerLighthouse> .\tee-check.ps1 --filter 0x06402/0/0 run tee-check
$ tee-enc/tee-check-0a

--- preflight ---
pass chain                                0x032
pass vault                                0x07c7080ad1d4f4385307b0780b3f9c3a3f15921
pass user (pays the Inco fee)             0x263840d28029c57e49578e46f721107ca37a88 0.83464544437643356 ETH
pass relay (gas only)                     0x25e6a340488f22a740df24c1c795ac3f919a825 0.80487876278819896 ETH
pass Inco executor deployed              0x6097110103506a4a0bf27c562620c1f5d2624
pass Inco fee                             0.000001 ETH per encrypted input

--- 1. encrypt the budget client-side ---
pass encrypted                           0.1 USDC in 56s
pass ciphertext is opaque                 2634 hex chars

--- 2. open a goal - ciphertext becomes an on-chain handle ---
pass openGoal confirmed                  2.43s gas 336605
pass goalId                             0x26a11cd9f506cc27ac823c2742b97ddc29a83ed6a9ff4f886f90788200800

The handle above is the whole point: it is what the chain stores instead
of 0.1. Reading it back tells you nothing.

--- 3. spend 0.04 - inside the encrypted budget ---
pass requestSpend confirmed              2.35s gas 296400
pass decision handle                     0xa85fe65187ba072c23578ba9e9f9f31c7985e07b05e7c1f7d449e850000
pass committed - and at this moment nobody, including this script, knows the answer
Retrying after transient error: [unknown] rpo error: code = Notfound desc = ciphertext for handle 0xa85fe65187ba072c23578ba9e9f9f31c7985e07b05e7c1f7d449e850000 not found, it might not have been processed yet, try again later
Retrying after transient error: [unknown] rpo error: code = Notfound desc = ciphertext for handle 0xa85fe65187ba072c23578ba9e9f9f31c7985e07b05e7c1f7d449e850000 not found, it might not have been processed yet, try again later
Retrying after transient error: [unknown] rpo error: code = Notfound desc = ciphertext for handle 0xa85fe65187ba072c23578ba9e9f9f31c7985e07b05e7c1f7d449e850000 not found, it might not have been processed yet, try again later
Retrying after transient error: [unknown] rpo error: code = Notfound desc = ciphertext for handle 0xa85fe65187ba072c23578ba9e9f9f31c7985e07b05e7c1f7d449e850000 not found, it might not have been processed yet, try again later
pass attestedReveal                      true in 10.55s (1 attempt), 2 signatures
pass decision is true
pass flipped plaintext is rejected         claimed false, on-chain verification refused it
pass finalIrreversible confirmed          3.54s gas 104017
pass on-chain isApproved agrees           true

--- 4. spend 0.09 - inside the cap, past the budget ---
Public cap is 4.0 and the payee is allowedListed, so nothing in
plaintext can refuse this. The enclave already debited 0.04,
leaving 0.06 - a figure this process never saw.
pass requestSpend confirmed              2.35s gas 296388
pass decision handle                     0x9a0f34ba5b30831a0742f720e5a0761a71983175e7e4ee0a0762d649c0000
pass committed - and at this moment nobody, including this script, knows the answer
Retrying after transient error: [unknown] rpo error: code = Notfound desc = ciphertext for handle 0x9a0f34ba5b30831a0742f720e5a0761a71983175e7e4ee0a0762d649c0000 not found, it might not have been processed yet, try again later
Retrying after transient error: [unknown] rpo error: code = Notfound desc = ciphertext for handle 0x9a0f34ba5b30831a0742f720e5a0761a71983175e7e4ee0a0762d649c0000 not found, it might not have been processed yet, try again later
Retrying after transient error: [unknown] rpo error: code = Notfound desc = ciphertext for handle 0x9a0f34ba5b30831a0742f720e5a0761a71983175e7e4ee0a0762d649c0000 not found, it might not have been processed yet, try again later
pass attestedReveal                      false in 8.25s (1 attempt), 2 signatures
pass decision is false
pass flipped plaintext is rejected         claimed true, on-chain verification refused it
pass finalIrreversible confirmed          1.95s gas 101392
pass on-chain isApproved agrees           false

--- 5. close the goal ---
pass closeGoal confirmed                 gas 30008

-----
Inco TEE round trip verified: encryption in, attested decryption out,
both decisions checked on chain, goal is closed.
https://sepolia.bloxscan.org/addresses/0x07c7080ad1d4f4385307b0780b3f9c3a3f15921
```

No USDC moves, the signer never starts, no vendor is contacted. Note the two flipped-plaintext assertions: a genuine attestation paired with the opposite claim is refused on chain.

06

Technologies, APIs and infrastructure used

None of the projects below is affiliated with this hackathon or with us. They were chosen because each solves a problem this design has, and each is named here with the specific guarantee it provides.

Technology	Version	Where it is used	Why it is load-bearing
Inco Lightning	1.0.2	PolicyVault.sol; browser-side encryption; attested reveal polling	Holds the budget as ciphertext and produces the attested decision verified on chain. Without it the budget is public and the agent can read it
Oasis ROFL	TDX	services/signer - RoflKeyStore, rofl.yaml, Dockerfile	Derives the per-goal payer key inside an attested enclave so no operator can extract it
x402	v1	packages/shared/src/x402; orchestrator client; self-hosted facilitator	The payment protocol the agent speaks. Its description field is also the attack surface the design exists to neutralise
Base Sepolia	chain 84532	PolicyVault, TraceAnchor, USDC settlement	Where policy state, approval records and settlement all live
Anthropic Claude	Opus 5	services/orchestrator/src/agent/llm.ts	The purchasing agent - and the component the demo attacks. The strongest model, so "it got fooled" is not a small-model excuse

6.1 Supporting stack

Layer	Technology
Contracts	Solidity 0.8.29, Foundry
Services	TypeScript, Node 22, viem, pnpm workspaces
Frontend	React, Vite, MetaMask
Settlement	USDC, EIP-3009 transferWithAuthorization, EIP-712
Verification	keccak256 hash chain, Merkle accumulator, standalone CLI verifier
CI	GitHub Actions - typecheck, Vitest, Foundry, import-boundary check

6.2 A note on the agent in these recordings

The orchestrator selects its agent at startup: Claude Opus 5 when an API key is present, and a deterministic stand-in otherwise. The runs captured in section 5 were recorded with the stand-in, which is why the console reports SCRIPTED. It reproduces the same compliance behaviour offline so the demo is reproducible without spending API credit. The distinction is immaterial to the security claim - the agent's verdict has no bearing on whether the spend is authorised, which is the entire point - but we would rather state it than have it noticed.

6.3 A note on Inco Lightning

Inco Lightning is **TEE-based, not FHE**. Its compute server runs inside a secure enclave; the browser encrypts to that enclave over HPKE, and the enclave signs attestations that `e.verifyDecryption` checks on chain. Inco's FHE-era package was `@inco/js`, which this project does not use anywhere. What Inco attests is precisely one thing: that a specific encrypted value decrypts to a specific result. It does not attest the AI's reasoning, our orchestrator's execution, or any enclave measurement of our own code - and we do not claim otherwise.

07

Stage 2 roadmap

Nothing below is a feature wish. Every item names a limit stated elsewhere in this document and says what closes it.

7.1 Returning unspent funds

- **Payer sweep-back on goal closure.** The ephemeral payer is funded per goal, and whatever is not spent stays at that address once the goal closes. Stage 2 returns the residue to the user's own wallet as part of `closeGoal`, so a goal is financially as well as logically complete.
- **Refund-on-timeout accounting.** A spend that is approved but never settles debits the encrypted budget permanently - the debit commits at request time and there is no path back. Closing this needs an encrypted credit-back, since the balance exists only as ciphertext.
- **Expiry reclamation.** A goal that expires with an unfinalized spend currently leaves the pending sequence occupied. Stage 2 adds an owner-callable reclaim that clears it and releases the payer balance.

7.2 Security hardening

- **Encrypt the remainder of the policy.** The per-call cap and call counter are public plaintext and the allowlist is a public mapping, so a vendor can read the ceiling and price just underneath it. Stage 2 moves the cap to an encrypted integer, the counter likewise, and allowlist membership to an encrypted predicate.
- **Authenticate the service surfaces.** The orchestrator and signer run permissive development CORS with no caller authentication. Nothing is exposed that the vault does not already bound, but a deployed instance needs an origin allowlist, per-caller rate limiting, and request authentication on both.
- **Deploy the ROFL enclave and publish the payer binding on Sapphire.** Custody is real today but not provable from Base. A Sapphire registry asserting that a payer address was derived inside an attested instance, checked by the trace verifier, turns an assertion into something a third party can confirm.
- **Anchor trace roots on chain.** `TraceAnchor` is written and tested but not yet called from the runtime, so a trace is verifiable only by someone holding the file. Anchoring the root at goal closure upgrades that to publicly committed.
- **Remove the signer as a trusted component.** The x402 exact scheme requires an EOA signature, which is the only reason an Authorization Signer exists at all. An escrow-based scheme variant makes the vault itself the payer and eliminates the last piece of code anyone has to trust.

7.3 Live model integration

- **Run the live model in the demo path.** The LLM agent is implemented and key-gated; Stage 2 makes Claude Opus 5 the default for recorded runs rather than the offline stand-in.
- **Surface the model's own reasoning in the console.** The summarised reasoning already travels the full length of the system - it reaches the event stream and is recorded in the trace - but the interface never renders it. That is a display gap, not a plumbing one, and it hides the single most persuasive artifact the demo produces: the attack landing at the model layer, in the model's own words.
- **Record model identity in the trace.** Pinning the model and version makes a run reproducible and lets an auditor tell an LLM decision from a scripted one without trusting the label in the interface.

7.4 The capability we most want to build

Delegated sub-budgets for sub-agents. A goal spawns child goals, each carrying its own encrypted sub-budget derived from the parent's without either being revealed. A coordinating agent could then hand spending authority to workers it dispatches - authority it cannot itself read, cannot raise, and cannot exceed in aggregate, because the parent budget bounds the sum of its children by construction.

This is the item we are most interested in because it is the one the primitive was already shaped for. The confidential check-and-debit generalises from one agent against one budget to a tree of agents against a tree of budgets without changing what is being proved. It is also where the architecture stops being a safety mechanism and becomes an organisational one: delegation with cryptographic limits is how you let a swarm of agents transact without any single compromise being unbounded - and the failure of one worker stays confined to the sub-budget it was given.

A

Proof of current student status

Submitted under the **Student Group** category. Both team members are currently enrolled students. Each institutional card below carries the holder's name, the issuing institution, and a validity date that has not yet passed.

Team member	Institution	Document
Adarsh Kumar	Indian Institute of Technology Patna	See A.1
Krishan Pratap Sharma	Indian Institute of Technology Bombay	See A.2

A.1 Adarsh Kumar - IIT Patna



A.2 Krishan Pratap Sharma - IIT Bombay

